
Tourism Hotel Booking System

Technical Design Report

Course: Software Engineering

Students: George Sleibi & Francis Lolas

University: Bethlehem University

Date: March 26, 2026

1. Introduction

The Tourism Hotel Booking System is a comprehensive backend platform designed to streamline hotel reservation management for tourism operators. The system enables guests to search for hotels, check room availability, create bookings, process payments, and receive notifications, while providing hotel managers and administrators with tools to manage hotel listings, room types, pricing rules, and booking policies.

Target Users

- **Guests:** Individuals seeking to book accommodations.
- **Hotel Managers:** Staff responsible for managing specific hotel properties.
- **System Administrators:** Personnel overseeing the entire platform.

Problem Addressed

Traditional hotel booking processes suffer from double-booking issues, manual pricing management, lack of real-time availability, and inefficient communication between guests and hotels.

Expected Benefits

- Real-time room availability checking to prevent overbooking.
- Dynamic pricing based on seasons and day types.
- Automated notifications reducing manual communication overhead.
- Centralized management for multiple hotels.
- Complete audit trail of bookings, payments, and notifications.

2. Domain Model

The system is built around six core entities that represent the real-world hotel booking domain. The domain model captures the complete lifecycle of a hotel booking: guests search available rooms, create bookings, make payments, and receive notifications, while managers control inventory and pricing.

Entity	Key Attributes	Relationships
Hotel	id, name, description, address, city, country, phone, email, starRating, active	One-to-many with RoomType
RoomType	id, name, description, capacity, basePrice, amenities, totalRooms, active	Many-to-one with Hotel; One-to-many with Booking, RoomAvailability

Booking	id, bookingReference, checkInDate, checkOutDate, numberOfGuests, numberOfRooms, totalPrice, status	Many-to-one with Guest and RoomType
Guest	id, fullName, email, phone, address	One-to-many with Booking
Payment	id, paymentReference, amount, paymentMethod, status, transactionId	Many-to-one with Booking
Notification	id, notificationReference, recipientEmail, type, status	Many-to-one with Booking

Supporting Entities

- **RoomAvailability:** Tracks daily room counts (total, booked, held, blocked).
- **PricingRule:** Defines dynamic pricing adjustments (weekday/weekend multipliers, seasonal rates).
- **BlockedDate:** Records dates when rooms are unavailable for maintenance.
- **User:** Manages authentication and authorization (admin, hotel manager, guest roles).

3. Module Boundaries (System Architecture)

The system follows a Modular Monolith architecture with six clearly separated modules, each designed to be potentially extractable into independent microservices in the future.

Module	Responsibility	Key Services
Auth	Authentication, authorization, user management	Login, registration, JWT generation, role management
Catalog	Hotel and room type management	CRUD for hotels and room types, search/filter
Availability & Pricing	Room availability tracking, price calculation	Availability checks, price calculation, room hold/release, blocked dates
Booking	Booking lifecycle management	Create, confirm, cancel, update bookings, search
Payment	Financial transaction processing	Create payment intent, process payments, refunds
Notification	Email and system notifications	Send confirmations, cancellations, payment alerts

Benefits of Modular Separation

- **Maintainability:** Each module has a single responsibility, making changes isolated and predictable.

- **Scalability:** Modules can be independently scaled or extracted to microservices when needed.
- **Testability:** Modules can be tested in isolation with mock dependencies.
- **Team Collaboration:** Multiple developers can work on different modules without merge conflicts.

4. Key Design Choices

Architectural Choice: Modular Monolith

The system begins as a single deployable unit with clear module boundaries. This approach provides the simplicity of a monolith during initial development while preserving the path to microservices. Each module's dependencies are explicitly managed, and modules communicate through well-defined interfaces, allowing future extraction with minimal refactoring.

Technology Stack

Layer	Technology	Rationale
Framework	Spring Boot 3.5	Industry standard, comprehensive ecosystem, built-in security and JPA
Security	Spring Security + JWT	Stateless authentication suitable for REST APIs and future microservices
Database	MySQL	Reliable relational database with strong ACID compliance for transactional integrity
API Documentation	OpenAPI 3.0 (Swagger)	Automatic documentation generation, interactive testing UI
Object Mapping	Lombok	Reduces boilerplate code while maintaining readability

Design Patterns

- **Repository Pattern:** Abstracts data access logic, enabling easy database changes.
- **Service Layer Pattern:** Encapsulates business logic, promotes separation of concerns.
- **DTO Pattern:** Prevents entity exposure, decouples API from internal domain.
- **Mapper Pattern:** Centralizes object transformation logic.
- **Strategy Pattern (in Pricing):** Enables flexible price calculation rules.
- **Factory Pattern (in Notifications):** Creates type-specific notification templates.

Scalability Considerations

- **Stateless Authentication:** JWT tokens allow horizontal scaling without session affinity.
- **Database Indexing:** Critical queries (availability checks, booking searches) are optimized with proper indexes.
- **Optimistic Locking:** Room availability updates use pessimistic write locks to prevent race conditions.
- **Caching Strategy:** Planned for frequently accessed data (hotel lists, pricing rules).

Key Trade-offs

Decision	Trade-off
Single database initially	Simple transactions but limits horizontal scaling at data layer.
Synchronous notifications	Guaranteed delivery but may slow booking flow (can be made async).
Mock payment	Rapid development but requires replacement for production.
Modular but monolith	Faster development than microservices but single deployment unit.

5. Conclusion

The Tourism Hotel Booking System delivers a complete hotel booking backend that addresses real-world requirements: preventing double-booking, enabling dynamic pricing, and automating guest communication. The modular monolith architecture provides immediate value while preserving long-term scalability options.

Key Strengths of the Design

- **Clean Separation:** Six independent modules with clear responsibilities enable parallel development and future microservices extraction.
- **Complete Business Flow:** End-to-end coverage from hotel management through availability, booking, payment, and notifications.
- **Production-Ready Security:** JWT-based authentication with role-based authorization (admin, hotel manager, guest).
- **Extensibility:** Pricing rules are data-driven, allowing dynamic configuration without code changes.

The system is ready for deployment as a monolithic application while being structurally prepared for the anticipated evolution into microservices. This balanced approach makes it suitable for both immediate business needs and long-term architectural goals.